

Yesterday 10:52 PM

as a beginner explain how to create a database in supabase and how to set it up in github. supabase is already connected to the repo. the premise is for customers. heres info: Project Description CareerConnect is a web-based platform designed to help job seekers manage their job search activities. The system allows users to create profiles, upload and manage resumes, search for job opportunities, track submitted applications, and follow the progress of their application process. The platform aims to centralize job-search activities and help users stay organized throughout their career development journey.

Show more

Absolutely. Since **Supabase is already connected to your GitHub repo**, the next step is to understand how the database itself works and then build the tables your CareerConnect application needs.

For a beginner, I would set it up in this order:

1. Understand what Supabase is doing
2. Create the database tables
3. Set up authentication
4. Connect your code to Supabase
5. Test registration/login
6. Build the profile/resume feature
7. Commit everything to GitHub

1. What you're actually building

Think of CareerConnect as having several types of information:

USER

└ Profile

└ Resume(s)

- └─ Job Applications
 - └─ Application Status
- └─ Job Opportunities

For Sprint 1, **don't build everything at once.**

Your recommended Sprint 1 database can start with:

```
profiles
resumes
```

Supabase already provides **authentication**, so you don't need to create your own password table.

Part 1 — Create the Supabase database

Go to your Supabase project and open:

Database → Tables

You'll create your tables there.

Table 1: `profiles`

This stores information about the customer/job seeker.

Create a table called:

```
profiles
```

I'd use these columns:

Column	Type	Purpose
<code>id</code>	<code>uuid</code>	Identifies the user
<code>first_name</code>	<code>text</code>	First name
<code>last_name</code>	<code>text</code>	Last name
<code>email</code>	<code>text</code>	Email
<code>phone</code>	<code>text</code>	Phone number
<code>location</code>	<code>text</code>	City/location

Column	Type	Purpose
bio	text	Short description
created_at	timestampz	Account creation date

Important: `id`

The `id` should correspond to the user's Supabase authentication account.

Supabase Auth has its own table of users:

```
auth.users
```

So conceptually:

```
auth.users
  |
  | user ID
  ↓
profiles
```

For example:

```
auth.users
-----
id: 1234-abcd
email: john@gmail.com

profiles
-----
id: 1234-abcd
first_name: John
last_name: Smith
location: Montreal
```

The two records belong to the same person because they have the same UUID.

Part 2 — Create the `profiles` table

In Supabase:

Database → **Tables** → **New Table**

Name:

profiles

Add:

id

Type:

uuid

Make it:

- Primary Key ☒
- Not nullable ☒

Then add:

first_name

Type:

text

Then:

last_name
text

Then:

email
text

Then:

phone
text

Then:

location
text

Then:

```
bio
text
```

Finally:

```
created_at
timestampz
```

For `created_at`, you can set the default value to:

```
SQL

now()
```

Part 3 — Create the resumes table

Create another table:

```
resumes
```

Use:

Column	Type	Purpose
<code>id</code>	<code>uuid</code>	Resume ID
<code>user_id</code>	<code>uuid</code>	Owner of resume
<code>file_name</code>	<code>text</code>	Name of uploaded file
<code>file_path</code>	<code>text</code>	Where file is stored
<code>created_at</code>	<code>timestampz</code>	Upload date

The important relationship is:

```
profiles
  |
  | id
  ↓
resumes
```

So if:

```
profiles.id = ABC123
```

then:

```
resumes.user_id = ABC123
```

means that resume belongs to that user.

Part 4 — Don't store the actual resume PDF in the database

This is a very important beginner concept.

You don't want to put the PDF itself inside a database row.

Instead, use:

Supabase Storage

Think of it like this:

DATABASE

```
resume ID
user ID
file name
file path
upload date
```

STORAGE

```
resume.pdf
resume2.pdf
resume3.pdf
```

The database tells you **where the file is**.

Supabase Storage actually stores the file.

So your `resumes` table might contain:

```
id: 789
user_id: 123
file_name: Mujdeh_Resume.pdf
file_path: resumes/123/Mujdeh_Resume.pdf
```

Part 5 — Create the Storage bucket

In Supabase go to:

Storage → New Bucket

Name it:

```
resumes
```

You can make it private.

For a career platform, **private is preferable** because resumes contain personal information.

Your structure will then be:

```
Supabase
├──
├── Authentication
├──
├── Database
│   ├── profiles
│   └── resumes
├──
└── Storage
    └── resumes
```

Part 6 — Authentication

For your Sprint 1 requirement:

User Registration/Login

You don't need to manually create a `users` table.

Go to:

Authentication → Providers

Enable:

Email

Supabase will handle:

Email
Password
Login
Registration
Session
User ID

Your application essentially says:

"Supabase, create an account for this customer."

Supabase handles the authentication.

Part 7 — How your GitHub project connects to Supabase

This is the part that can initially be confusing.

Your GitHub repository contains your **code**.

Supabase contains your **database**.

They're connected through your application's Supabase credentials.

For example:

```
GitHub Repository
  |
  | JavaScript / React
  ↓
Supabase Client
  |
  ↓
Supabase
┌───────────┐
| Authentication |
| Database       |
└───────────┘
```

| Storage |

Your code will have something similar to:

JavaScript

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  SUPABASE_URL,
  SUPABASE_KEY
)
```

Part 8 — NEVER put your Supabase secret directly in GitHub

This is extremely important.

You should **not** do this:

JavaScript

```
const supabase = createClient(
  "https://something.supabase.co",
  "my-secret-key"
)
```

Instead, use environment variables.

Create a file:

.env

For example:

env

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key
```

Depending on your framework, the variable names may be different.

For a Vite React project, `VITE_` is commonly used.

Then your code can do:

JavaScript

```
const supabase = createClient(  
  import.meta.env.VITE_SUPABASE_URL,  
  import.meta.env.VITE_SUPABASE_ANON_KEY  
)
```

Part 9 — Add `.env` to `.gitignore`

You do not want `.env` uploaded to GitHub.

Your `.gitignore` should contain:

```
.env  
.env.local  
.env.*.local
```

Then GitHub won't track your local environment variables.

Your team members will each have their own `.env` file.

Part 10 — How your teammates get the Supabase connection

This is where your team's GitHub workflow comes in.

Someone clones the repository:

Bash

```
git clone YOUR_REPOSITORY_URL
```

Then:

Bash

```
npm install
```

But their computer doesn't automatically have your `.env`.

So your team should have something like:

```
.env.example
```

which is **safe to commit**.

Example:

```
env
```

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_ANON_KEY=
```

Then your README can say:

1. Clone the repository
2. Run `npm install`
3. Create a `.env` file
4. Copy the values from `.env.example`
5. Add the Supabase project credentials
6. Run `npm run dev`

Part 11 — Your GitHub folder structure

For your CareerConnect project, I'd recommend something like:

```
CareerConnect/  
|  
├─ src/  
|   ├─ components/  
|   ├─ pages/  
|   ├─ services/  
|   │   └─ supabase.js  
|   ├─ App.jsx  
|   └─ main.jsx  
|  
├─ public/  
|  
├─ .env.example  
├─ .gitignore  
├─ package.json  
├─ README.md  
|  
└─ documentation/  
    ├─ meeting-minutes/  
    ├─ ai-logs/  
    ├─ sprint-planning/  
    └─ requirements/
```

The exact structure depends on what framework your team was given, but this is a reasonable starting point for React/Vite.

Part 12 — Create the Supabase client

Install Supabase in your project:

Bash

```
npm install @supabase/supabase-js
```

Then create:

```
src/services/supabase.js
```

Put:

JavaScript

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient(
  supabaseUrl,
  supabaseAnonKey
)
```

Now other files can import it:

JavaScript

```
import { supabase } from '../services/supabase'
```

Part 13 — Registration

Now you can actually use Supabase.

For example:

JavaScript

```
const { data, error } = await supabase.auth.signUp({
  email: email,
  password: password
})
```

Conceptually:

Customer enters:

Email: john@gmail.com

Password: *****

↓

Registration page

↓

supabase.auth.signUp()

↓

Supabase Authentication

↓

Account created

Part 14 — Login

Login is similar:

JavaScript

```
const { data, error } =
  await supabase.auth.signInWithPassword({
    email: email,
    password: password
  })
```

If successful, Supabase gives your application information about the logged-in user.

You can get the current user:

JavaScript

```
const {
  data: { user }
} = await supabase.auth.getUser()
```

Then:

JavaScript

```
user.id
```

gives you the user's UUID.

That UUID is extremely important because you use it when working with their profile and resumes.

Part 15 — Creating a profile

After registration, you could create their profile:

JavaScript

```
const {
  data: { user }
} = await supabase.auth.getUser()

const { error } = await supabase
  .from('profiles')
  .insert({
    id: user.id,
    first_name: firstName,
    last_name: lastName,
    email: user.email
  })
```

So you now have:

Authentication

```
user.id
```

```
|
```

```
|
```

```
▼
```

```
profiles.id
```

Part 16 — Getting a customer's profile

Suppose John logs in.

Your code gets:

JavaScript

```
user.id
```

Then:

JavaScript

```
const { data, error } = await supabase
  .from('profiles')
  .select('*')
  .eq('id', user.id)
  .single()
```

Now `data` contains John's profile.

For example:

JavaScript

```
console.log(data.first_name)
```

could give:

```
John
```

Part 17 — VERY important: Row Level Security

Once you're dealing with customer data, you need to think about security.

You don't want:

```
John
↓
somehow accessing
↓
Sarah's resume
```

Supabase uses **Row Level Security (RLS)** for this.

For your `profiles` table, the basic idea is:

A logged-in user can only access their own profile.

For example, the policy can be based on:

SQL

```
auth.uid() = id
```

For `resumes`:

SQL

```
auth.uid() = user_id
```

That means:

```
John's account  
auth.uid() = John
```

```
John's resume  
user_id = John
```

↓

MATCH 

But:

```
John's account  
auth.uid() = John
```

```
Sarah's resume  
user_id = Sarah
```

↓

NO MATCH 

This is something your team should set up before considering the customer data feature finished.

Part 18 — Your Sprint 1 database

I wouldn't create 15 tables just because your project eventually needs them.

For Sprint 1, I'd keep it simple:

```
graph TD
  AUTH --- users
  users --- profiles
  users --- resumes
```

Later sprints can add:

```
job_opportunities
applications
application_status_history
saved_jobs
```

Eventually:

```
graph LR
  users --- profiles
  profiles --- resumes
  profiles --- applications
  applications --- job_opportunities
  applications --- status_history
```

That's much easier for your team to understand and develop incrementally.

Part 19 — What I would actually implement for your Sprint 1 demo

Your instructions specifically recommend:

Feature 1

User Registration/Login

Feature 2

Resume Upload OR User Profile Management

I'd choose:

Registration/Login + Profile Management + basic Resume Upload

if your team has enough time.

The demo could look like:

Screen 1 — Register

CareerConnect

Create your account

Email: [_____]

Password: [_____]

[Register]



Supabase creates the account.



Screen 2 — Profile

My Profile

First Name: Mujdeh

Last Name: _____

Email: _____

Location: Montreal

Bio: _____

[Save Profile]



Information gets stored in:

profiles



Screen 3 — Resume

My Resumes

Resume.pdf



PDF goes into:

Supabase Storage

while information about it goes into:

resumes

That would give you a very clear Sprint 1 demonstration.

Your Git workflow

Since you're working as a team, don't have everyone directly modify `main`.

A simple workflow:

```
main
|
├─ feature/login
|
├─ feature/profile
|
└─ feature/resume-upload
```

For example, someone working on registration:

Bash

```
git checkout -b feature/registration
```

Work on the feature:

Bash

```
git add .
git commit -m "Add user registration"
```

Then:

Bash

```
git push origin feature/registration
```

Then open a **Pull Request on GitHub**.

Another team member reviews it.

If everything looks good:

```
graph TD
    A[Pull Request] --> B[Code Review]
    B --> C[Approved]
    C --> D[Merge into main]
```

This fits directly with the **branching strategy + pull request + code review** requirements in your Sprint 1 instructions.

One important distinction

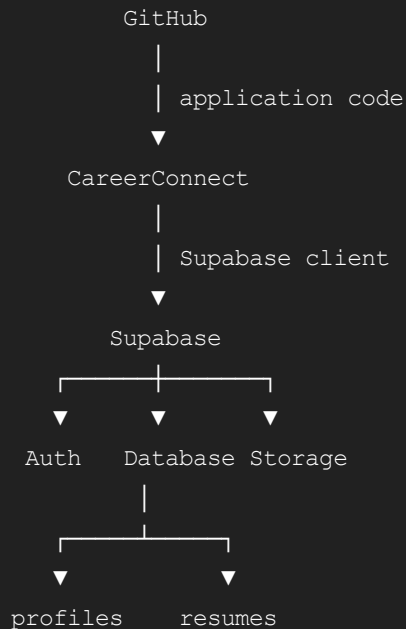
Your GitHub repository **doesn't contain your Supabase database** in the normal sense.

You have:

```
graph TD
    subgraph GitHub
        SC[source code]
        README[README]
        DOC[documentation]
        PKG[package.json]
        ENV[.env.example]
    end

    subgraph Supabase
        subgraph PostgreSQL_Database [PostgreSQL Database]
            PRO[profiles]
            RES1[resumes]
        end
        subgraph Storage
            RES2[resumes]
        end
        AUTH[Authentication]
    end
```

Your application connects the two:



For your specific Sprint 1, I'd recommend doing the database in this order:

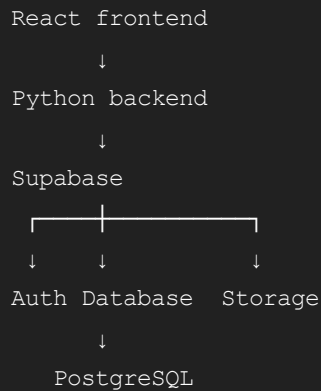
1. Create `profiles`
2. Connect `profiles.id` to the authenticated user's ID
3. Turn on RLS and create the "user can only access their own profile" policy
4. Create the `resumes` table
5. Create the private `resumes` Storage bucket
6. Add Supabase client to your project
7. Add `.env` and `.env.example`
8. Implement registration/login
9. Implement profile creation/editing
10. Implement resume upload
11. Test everything with a test account
12. Commit/push through a feature branch and PR

If you tell me **what framework/language your GitHub project is using (React, React Native, Next.js, plain HTML/JS, etc.)**, I can walk you through the **exact clicks in Supabase + exact files/code to create**, starting from a completely empty database.

using react fro frontened, python for backend

Perfect. With **React frontend + Python backend**, I would change the setup slightly from the previous example.

The architecture should be:



The most important beginner concept is: **React should handle what the customer sees; Python should handle your application logic; Supabase handles the database, authentication, and file storage.**

For Sprint 1, I'd build **Registration/Login + Profile Management** first, then add resume upload if you have time.

Recommended project structure

```
CareerConnect/
|
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   ├── pages/
|   |   └── App.jsx
|   ├── package.json
|   └── .env
|
├── backend/
|   ├── app/
|   |   ├── routes/
|   |   ├── services/
|   |   ├── models/
|   |   └── main.py
|   ├── requirements.txt
|   └── .env
|
├── documentation/
|   ├── meeting-minutes/
|   ├── ai-logs/
|   └── sprint-planning/
|
├── README.md
└── .gitignore
```

The exact Python structure depends on whether your team is using **Flask** or **FastAPI**.

If you haven't chosen yet, **FastAPI** is a very straightforward choice for a React + Python project.

Database

For your first Sprint, create:

```
auth.users
  |
  | id
  ↓
profiles
  |
  | id
  ↓
resumes
```

Actually, I would make the relationship:

```
auth.users
  |
  |
  ▼
profiles
  |
  |
  ▼
resumes
```

with:

```
profiles
-----
id          UUID
first_name  TEXT
last_name   TEXT
email       TEXT
phone       TEXT
location    TEXT
bio         TEXT
created_at  TIMESTAMP
```

and:

```
resumes
-----
id          UUID
user_id     UUID
file_name   TEXT
```

file_path	TEXT
created_at	TIMESTAMP

How the three technologies communicate

Suppose a customer enters:

```
Email: jane@gmail.com
Password: 123456
```

React displays the registration form.

React sends the information to your Python backend:

```
React
  ↓
POST /register
  ↓
Python
  ↓
Supabase
```

Supabase creates the authentication account.

Then your Python backend can create the customer's profile:

```
profiles

id: ABC123
first_name: Jane
last_name: Smith
email: jane@gmail.com
```

Later, when Jane logs in:

```
React
  ↓
Python
  ↓
Supabase
  ↓
Jane's data
  ↓
Python
  ↓
React
```

React then displays:

```
Welcome, Jane!
```

```
My Profile
```

```
-----
```

```
Jane Smith
```

```
Montreal
```

```
jane@gmail.com
```

One thing I would NOT do

Don't have React directly contain your **Supabase service-role key**.

Your backend should have access to sensitive credentials.

Your `.gitignore` should contain:

```
.env
.env.local
__pycache__/
venv/
.venv/
node_modules/
```

And commit an example:

```
.env.example
```

For example, your backend `.env` could contain:

```
env
```

```
SUPABASE_URL=your_supabase_url
SUPABASE_KEY=your_backend_key
```

while the actual `.env` stays out of GitHub.

If you're starting from scratch, **the next thing I'd do is set up the actual Supabase database first**, then connect it to your Python backend, then connect React to Python. I can walk you through **every click and file, including exactly what to put into Supabase's SQL Editor, what each line means, and how to test it**, assuming you're a complete beginner.